



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251182
Nama Lengkap	Michael Dylan
Minggu ke / Materi	12 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU 1 (40%)

12.3.1 Dictionary sebagai set penghitung (counters)

Kalau kita disuruh menghitung jumlah huruf yang muncul di dalam string, ada beberapa cara yang bisa dipakai. Bisa pakai banyak variabel untuk tiap huruf, bisa juga pakai list dengan indeks angka, atau paling praktis pakai dictionary. Dictionary lebih enak dipakai karena kita nggak perlu nyiapin semua huruf dari awal. Tinggal simpan huruf yang muncul sebagai key dan jumlah kemunculannya sebagai value. Jadi proses perhitungannya lebih simpel, fleksibel, dan mudah dipahami dibanding cara lainnya. Contohnya seperti di bawah ini.

```
word = 'brontosaurus'
```

```
d = dict()
```

```
for c in word:
```

```
    if c not in d:
```

```
        d[c] = 1
```

```
    else:
```

```
        d[c] = d[c] + 1
```

```
print(d)
```

Output program di atas adalah:

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Dari hasil itu terlihat kalau huruf "a" dan "b" muncul 1 kali, huruf "o" muncul 2 kali, dan huruf lainnya juga dihitung sesuai banyak kemunculannya dalam string. Model perhitungan tadi disebut histogram, yaitu cara untuk menghitung frekuensi atau jumlah kemunculan data. Perulangan for dipakai untuk mengecek tiap karakter dalam string. Kalau karakter belum ada di dictionary, maka dibuat key baru dengan nilai awal 1. Tapi kalau karakter itu sudah ada, nilainya langsung ditambah sesuai jumlah kemunculannya. Dictionary punya method `get()` yang digunakan untuk mengambil nilai berdasarkan key tertentu. Kalau key tersebut ada di dictionary, maka nilainya akan ditampilkan. Tapi kalau key tidak ada, maka akan mengembalikan nilai default yang sudah ditentukan. Sebagai Contoh:

```
>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
```

```
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0
```

Method `get()` membuat penulisan loop histogram jadi lebih sederhana karena otomatis menangani key yang belum ada di dictionary, sehingga kode bisa diringkas dari beberapa baris menjadi satu baris saja.

```
word = 'brontosaurus'
d = dict()
for c in word:
    d[c] = d.get(c,0) + 1
print(d)
```

Output program di atas adalah:

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Penggunaan method `get()` untuk menyederhanakan loop perhitungan sudah menjadi cara yang umum dipakai di Python karena hasilnya sama seperti memakai `if` dan `in`, tetapi penulisannya lebih singkat dan praktis.

12.3.2 Dictionary dan File

Salah satu fungsi dictionary yang sering dipakai adalah untuk menghitung jumlah kemunculan kata dalam sebuah file teks. Contohnya menggunakan teks sederhana dari cerita Romeo dan Juliet, mulai dari versi tanpa tanda baca sampai versi yang menggunakan tanda baca lengkap.

But soft what light through yonder window breaks

It is the east and Juliet is the sun

Arise fair sun and kill the envious moon

Who is already sick and pale with grief

Program Python dibuat untuk membaca tiap baris pada file, memisahkan kata-katanya, lalu menghitung jumlah kemunculan setiap kata menggunakan dictionary. Proses ini memakai nested loop, yaitu perulangan luar untuk membaca baris file dan perulangan dalam untuk mengecek setiap kata, sehingga semua kata dalam file bisa dihitung satu per satu. Contohnya di bawah ini:

```

fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

```

```

counts = dict()
for line in fhand:
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1
print(counts)

```

Output program di atas adalah:

```

python count1.py
Enter the file name: romeo.txt
{'and': 3, 'envious': 1, 'already': 1, 'fair': 1,
'is': 3, 'through': 1, 'pale': 1, 'yonder': 1,
'what': 1, 'sun': 2, 'Who': 1, 'But': 1, 'moon': 1,
'window': 1, 'sick': 1, 'east': 1, 'breaks': 1,
'grief': 1, 'with': 1, 'light': 1, 'It': 1, 'Arise': 1,
'kill': 1, 'the': 3, 'soft': 1, 'Juliet': 1}

```

Pada bagian else, penambahan nilai variabel ditulis lebih singkat memakai +=, yang artinya sama dengan menambahkan nilai sebelumnya secara manual. Selain +=, ada juga operator lain seperti -=, *= dan /=. Saat program dijalankan, hasil jumlah kata akan ditampilkan dalam urutan hash yang belum terurut.

12.3.3 Looping dan Dictionary

Pada perulangan for, dictionary akan menelusuri semua key yang ada di dalamnya, lalu mencetak setiap key beserta nilai yang berhubungan dengannya.

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}  
for key in counts:  
    print(key, counts[key])
```

Output program di atas adalah:

```
jan 100  
chuck 1  
annie 42
```

Output dictionary tidak ditampilkan dalam urutan tertentu. Dengan menggunakan berbagai bentuk looping pada dictionary, kita bisa mencari data tertentu, misalnya menampilkan semua key yang memiliki nilai lebih dari 10. Jadi programnya seperti di bawah ini:

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}  
for key in counts:  
    if counts[key] > 10 :  
        print(key, counts[key])
```

Output program di atas adalah:

```
jan 100  
annie 42
```

Program diatas hanya akan menampilkan data nilai diatas 10. Pada perulangan for yang menelusuri key pada dictionary, digunakan operator indeks untuk mengambil nilai dari setiap key sehingga key dan nilainya bisa ditampilkan bersama. Untuk menampilkan key dictionary secara alfabet, pertama key diubah dulu menjadi list menggunakan method `keys()`. Setelah itu list diurutkan dengan `sort()`, lalu dilakukan perulangan untuk mencetak setiap key beserta nilainya sesuai urutan alfabet. Kodenya seperti di bawah ini:

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}  
lst = list(counts.keys())  
print(lst)
```

```
lst.sort()
for key in lst:
    print(key, counts[key])
```

Output program di atas adalah:

```
['jan', 'chuck', 'annie']
annie 42
chuck 1
jan 100
```

Awalnya key pada dictionary ditampilkan dalam keadaan tidak terurut dari method `keys()`. Setelah itu, key dan nilainya ditampilkan lagi menggunakan `for` setelah diurutkan terlebih dahulu.

12.3.4 Advanced Text Parsing

Pada bagian ini digunakan file romeo.txt yang masih memiliki tanda baca lengkap, berbeda dengan contoh sebelumnya yang tanda bacanya sudah dihapus.

But, soft! what light through yonder window breaks?

It is the east, and Juliet is the sun .

Arise, fair sun and kill the envious moon ,

Who is already sick and pale with grief ,

Function split() di Python memisahkan kata berdasarkan spasi, sehingga kata yang memiliki tanda baca seperti "soft!" dan "soft" dianggap berbeda. Hal yang sama juga terjadi pada huruf besar dan kecil, misalnya "who" dan "Who" dihitung terpisah. Untuk mengatasinya bisa memakai method string seperti lower(), punctuation, dan translate().

```
line.translate(str.maketrans(fromstr, tostr, deletestr))
```

Method translate() digunakan untuk mengganti atau menghapus karakter tertentu dalam string. Pada kasus ini, parameter deletetr dipakai untuk menghapus semua tanda baca, sehingga Python otomatis mengenali karakter mana yang termasuk tanda baca.

```
>>> import string
>>> string.punctuation
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

Penggunaan dalam program sebagai berikut:

```
import string

fname = input('Enter the file name: ')

try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()
for line in fhand:
    line = line.rstrip()
    line = line.translate(line.maketrans("", "", string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1
print(counts)
```

Output program di atas adalah:

```
Enter the file name: romeo-full.txt
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2,
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Link Github = https://github.com/MichaelDylan-ti/71251182_michaeldylan/tree/main

LATIHAN 12.1

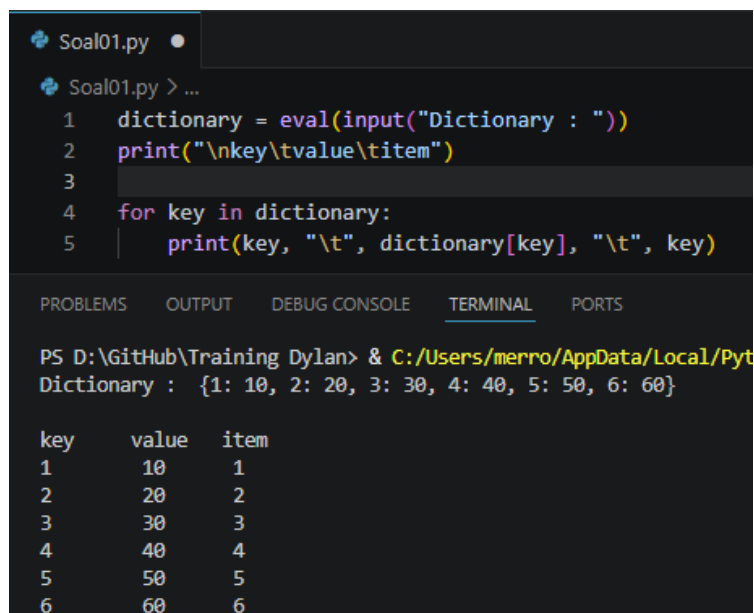
Bualah sebuah program untuk mendapatkan nilai key, value, dan item dari sebuah dictionary.

Contoh:

Dictionary : {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}

Output:	key	value	item
	1	10	1
	2	20	2
	3	30	3
	4	40	4
	5	50	5
	6	60	6

JAWABAN:



```
Soal01.py •
Soal01.py > ...
1 dictionary = eval(input("Dictionary : "))
2 print("\nkey\tvalue\titem")
3
4 for key in dictionary:
5     print(key, "\t", dictionary[key], "\t", key)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Pyt
Dictionary : {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}

key	value	item
1	10	1
2	20	2
3	30	3
4	40	4
5	50	5
6	60	6

Gambar 12.1: Program Input dan Output dari Latihan 12.1. Sumber Python Vscode

Program ini digunakan untuk menampilkan key, value, dan item dari sebuah dictionary yang di-input langsung oleh user. Input dictionary dimasukkan menggunakan input() lalu dibaca dengan eval() supaya data yang dimasukkan bisa dianggap sebagai dictionary Python. Setelah itu program melakukan perulangan for key in dictionary untuk mengambil semua key yang ada di dalam dictionary. Nilai value diambil menggunakan dictionary[key], kemudian program mencetak key, value, dan item dalam bentuk tabel sederhana sesuai data yang dimasukkan user.

LATIHAN 12.2

Bualah sebuah program untuk memetakan dua list mejadi satu dictionary.

Contoh:

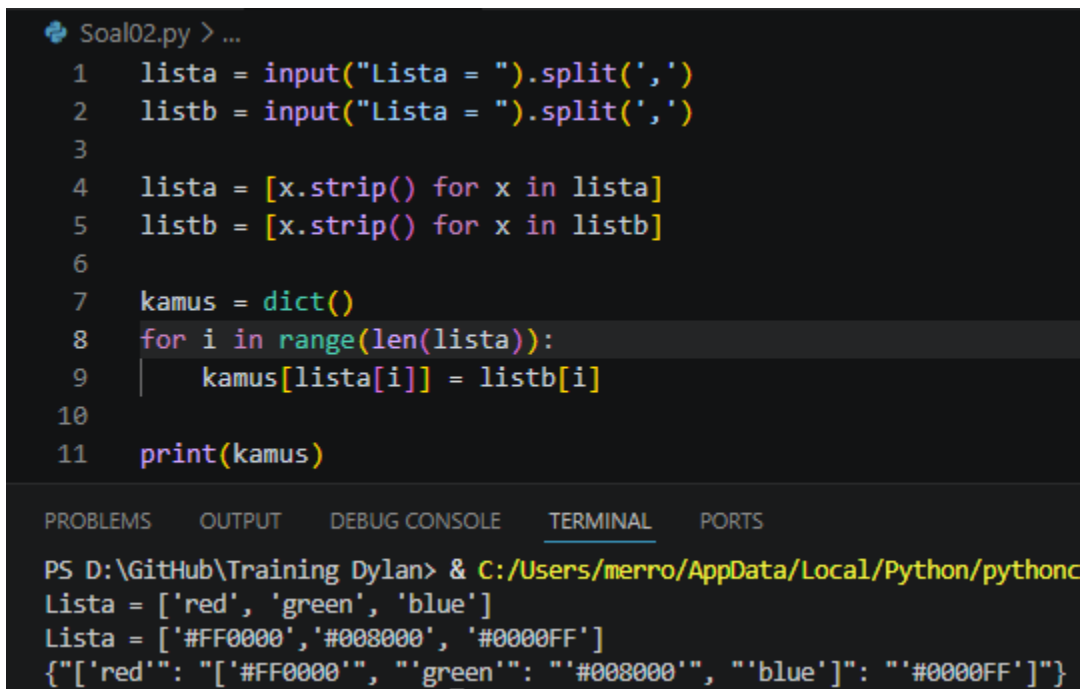
Data List

Lista = ['red', 'green', 'blue']

Listb = ['#FF0000', '#008000', '#0000FF']

Output {'green': '#008000', 'blue': '#0000FF', 'red': '#FF0000'}

JAWABAN:



```
Soal02.py > ...
1  lista = input("Lista = ").split(',')
2  listb = input("Lista = ").split(',')
3
4  lista = [x.strip() for x in lista]
5  listb = [x.strip() for x in listb]
6
7  kamus = dict()
8  for i in range(len(lista)):
9      kamus[lista[i]] = listb[i]
10
11 print(kamus)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythonc
Lista = ['red', 'green', 'blue']
Listb = ['#FF0000', '#008000', '#0000FF']
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

Gambar 12.2: Program Input dan Output dari Latihan 12.2. Sumber Python Vscode

Program ini menerima input dua buah data berupa string yang dipisahkan koma, yaitu `lista` dan `listb`. Setelah itu, masing-masing input diubah menjadi list dengan `.split(',')`, lalu setiap elemennya dibersihkan dari spasi pakai `strip()` supaya tidak ada spasi yang mengganggu. Program kemudian membuat dictionary kosong bernama `kamus`, lalu menjalankan perulangan berdasarkan panjang `lista`, di mana setiap elemen `lista[i]` dijadikan key dan `listb[i]` dijadikan value di dalam dictionary. Jadi intinya program ini menggabungkan dua list menjadi pasangan key-value. Output akhirnya berupa dictionary yang menampilkan pasangan data sesuai urutan input, misalnya nama warna sebagai key dan kode warna sebagai value, sehingga hasil akhirnya seperti penggabungan dua list menjadi satu kamus data.

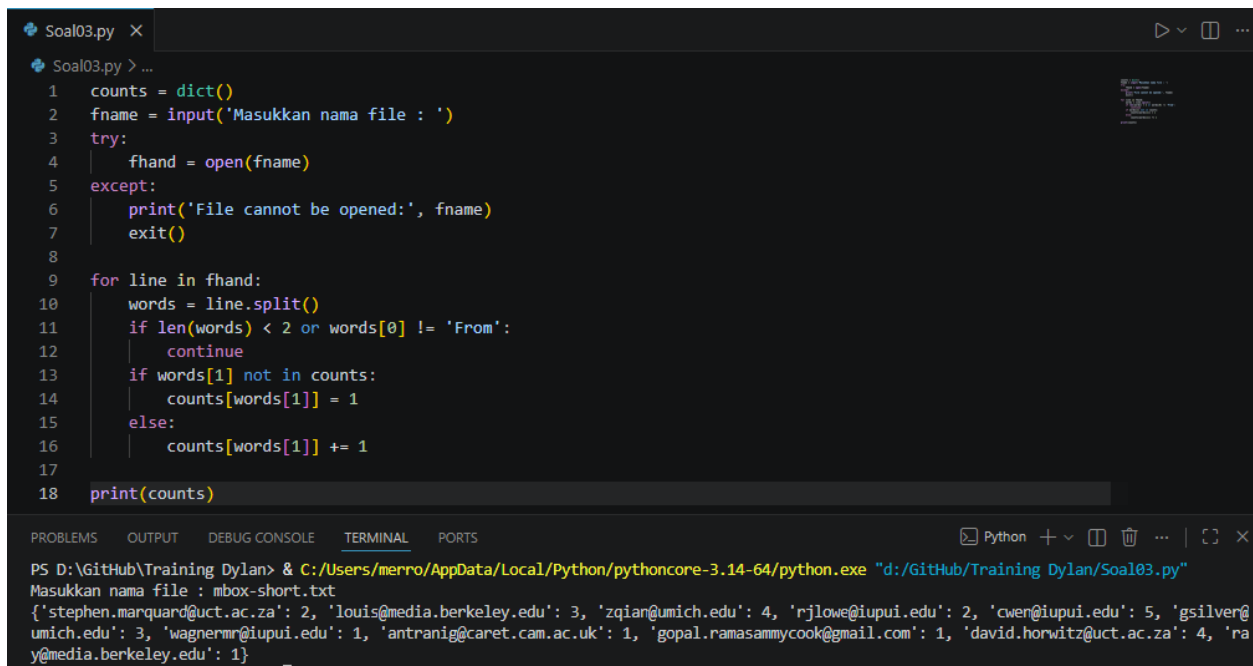
LATIHAN 12.3

Dengan menggunakan file **mbox-short.txt**, buatlah program yang dapat membaca log email dan sajikan dalam histogram menggunakan dictionary. Kemudian hitung berapa banyak pesan yang masuk dari email dan sajikan dalam bentuk dictionary. Silakan cek bagian dibawah ini untuk contoh output dari programnya.

Masukkan nama file : `mbox-short.txt`

```
{'gopal.ramasammycook@gmail.com': 1, 'louis@media.berkeley.edu': 3, 'cwen@iupui.edu': 5, 'antranig@caret.cam.ac.uk': 1, 'rjlowe@iupui.edu': 2, 'gsilver@umich.edu': 3, 'david.horwitz@uct.ac.za': 4, 'wagnermr@iupui.edu': 1, 'zqian@umich.edu': 4, 'stephen.marquard@uct.ac.za': 2, 'ray@media.berkeley.edu': 1}
```

JAWABAN:



```
Soal03.py X
Soal03.py > ...
1 counts = dict()
2 fname = input('Masukkan nama file : ')
3 try:
4     fhand = open(fname)
5 except:
6     print('File cannot be opened:', fname)
7     exit()
8
9 for line in fhand:
10     words = line.split()
11     if len(words) < 2 or words[0] != 'From':
12         continue
13     if words[1] not in counts:
14         counts[words[1]] = 1
15     else:
16         counts[words[1]] += 1
17
18 print(counts)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [] ... | [ ] X

PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-64/python.exe "d:/GitHub/Training Dylan/Soal03.py"
Masukkan nama file : mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnermr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}
```

Gambar 12.3: Program Input dan Output dari Latihan 12.3. Sumber Python Vscod

Program ini meminta input berupa nama file, yaitu mbox-short.txt, lalu program mencoba membuka file tersebut. Kalau file tidak bisa dibuka, program akan menampilkan pesan error “File cannot be opened” dan berhenti. Setelah file berhasil dibuka, program membaca isi file per baris, lalu setiap baris dipecah menjadi kata-kata dengan split(). Program hanya memproses baris yang diawali kata “From” dan memiliki minimal 2 kata, sedangkan baris lain akan dilewati pakai continue. Dari baris yang valid itu, program mengambil kata kedua (words[1]) sebagai data utama, lalu menghitung berapa kali masing-masing email muncul menggunakan dictionary counts (kalau belum ada maka diisi 1, kalau sudah ada ditambah 1). Output akhirnya adalah dictionary yang berisi daftar email sebagai key dan jumlah kemunculannya sebagai value, jadi hasilnya menunjukkan frekuensi kemunculan setiap email di dalam file tersebut.

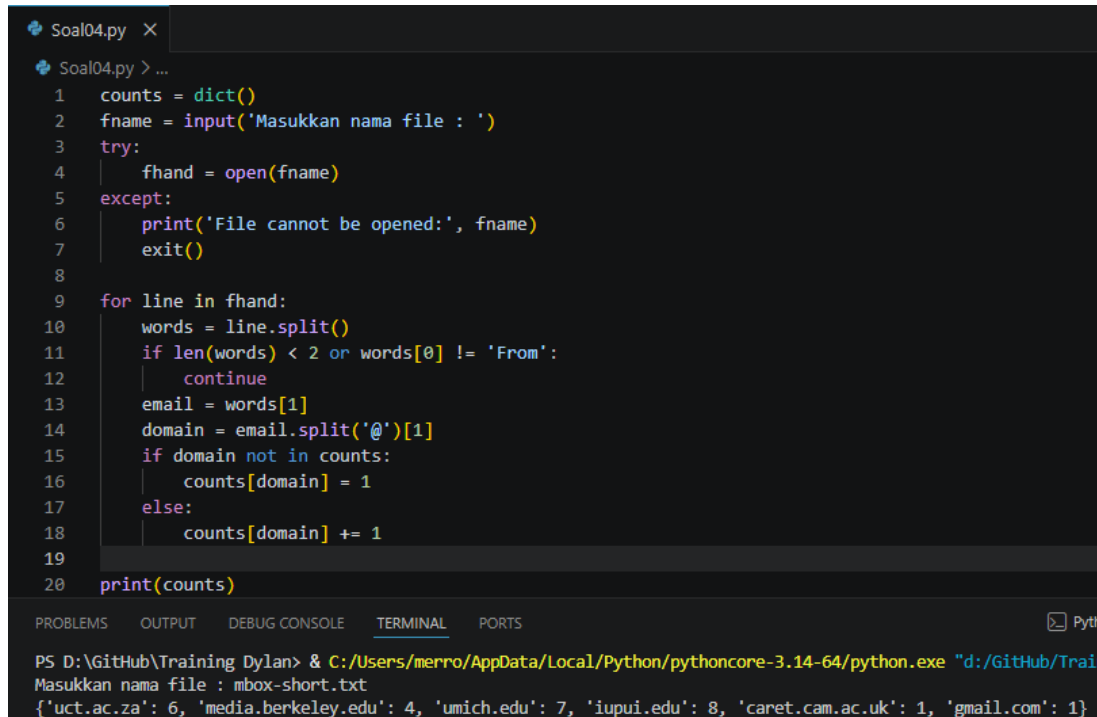
LATIHAN 12.4

Dengan menggunakan file mbox-short.txt, buat program untuk mencatat data nama domain pengirim pesan. Hitunglah jumlah pesan yang dikirim masing-masing domain. sajikan dalam bentuk dictionary. Silakan cek bagian dibawah ini untuk contoh output dari programnya.

Masukkan nama file: mbox-short.txt

```
{'media.berkeley.edu': 4, 'uct.ac.za': 6, 'umich.edu': 7, 'gmail.com': 1, 'caret.cam.ac.uk': 1, 'iupui.edu': 8}
```

JAWABAN:



```
Soal04.py x
Soal04.py > ...
1 counts = dict()
2 fname = input('Masukkan nama file : ')
3 try:
4     fhand = open(fname)
5 except:
6     print('File cannot be opened:', fname)
7     exit()
8
9 for line in fhand:
10     words = line.split()
11     if len(words) < 2 or words[0] != 'From':
12         continue
13     email = words[1]
14     domain = email.split('@')[1]
15     if domain not in counts:
16         counts[domain] = 1
17     else:
18         counts[domain] += 1
19
20 print(counts)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-64/python.exe "d:/GitHub/Trail
Masukkan nama file : mbox-short.txt
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}

Gambar 12.4: Program Input dan Output dari Latihan 12.4. Sumber Python Vscode

Program ini menerima input berupa nama file, yaitu mbox-short.txt, lalu program mencoba membuka file tersebut. Kalau file tidak ditemukan atau gagal dibuka, program akan menampilkan pesan “File cannot be opened” dan langsung berhenti. Setelah file berhasil dibuka, program membaca isi file baris per baris, lalu setiap baris dipecah menjadi kata-kata dengan split(). Program hanya memproses baris yang diawali kata “From” dan memiliki minimal 2 kata, sedangkan baris lain akan dilewati. Dari baris yang valid, program mengambil email pada posisi words[1], kemudian memisahkan domain email dengan cara split menggunakan tanda @ dan mengambil bagian setelahnya. Domain tersebut lalu dihitung kemunculannya menggunakan dictionary counts, jika domain belum ada maka diisi 1, kalau sudah ada maka nilainya ditambah 1. Output akhirnya berupa dictionary yang menampilkan setiap domain email sebagai key dan jumlah kemunculannya sebagai value, sehingga hasilnya menunjukkan frekuensi domain email yang muncul di dalam file tersebut.